Université d'Évry – Université Paris-Saclay

Master 2 CNS – SR

Software Defined Network

Projet 03

Nom : Atsamnia
Prénom : Abderraouf
N° étudiant : 20264731
Parcours : CNS – SR

Octobre 2026

Table des matières

1 Introduction **2**

2 Préparation **2**

3 Exercice 1 – Premier déploiement **2**

4 Exercice 2 – Passerelle avec POX **5**

4.1 Les deux parties du script 5

4.2 Exécution du script d'origine 5

4.3 POX comme passerelle 6

5 Exercice 3 – Routage multi-chemins **9**

5.1 La topologie 9

5.2 Fonctionnement de `tp.py` (version d'origine) 10

5.3 Le problème observé 11

5.4 La solution 12

5.5 Tests 13

5.6 Limites 16

6 Modifications des fichiers fournis **16**

7 Bilan **17**

A Code de `tp.py` **17**

B Modifications de `diamond.py` et `passerelle.py` **21**

1 Introduction

Ce TP utilise le contrôleur POX, écrit en Python, pour programmer le comportement de switches OpenFlow. Il comporte trois exercices : une première prise en main avec le composant `l2_learning` (exercice 1), une passerelle IPv4 programmée dans POX avec `l3_learning` (exercice 2), et un composant de routage multi-chemins écrit à la main pour une topologie en losange (exercice 3).

Les réponses aux questions sont dans des encadrés, avec les captures d'écran correspondantes (S01 à S22). Le code complet de la solution est en annexe et dans l'archive jointe.

Environnement.

- Machine hôte Windows avec VMware Workstation, connexion à la VM en SSH.
- VM Mininet 2.3.0 sous Ubuntu 20.04, Python 3.8.
- POX 0.7.0 (*gar*). Le sujet cite la version 0.2.0 (*carp*), qui ne fonctionne qu'avec Python 2.

Quelques commandes ou fichiers du sujet ont dû être adaptés. Ces adaptations sont listées dans la section 6.

2 Préparation

Avant de commencer, j'ai vérifié la version de Python, la présence du dossier `~/pox` et des contrôleurs, et la version de Mininet (figure 1).

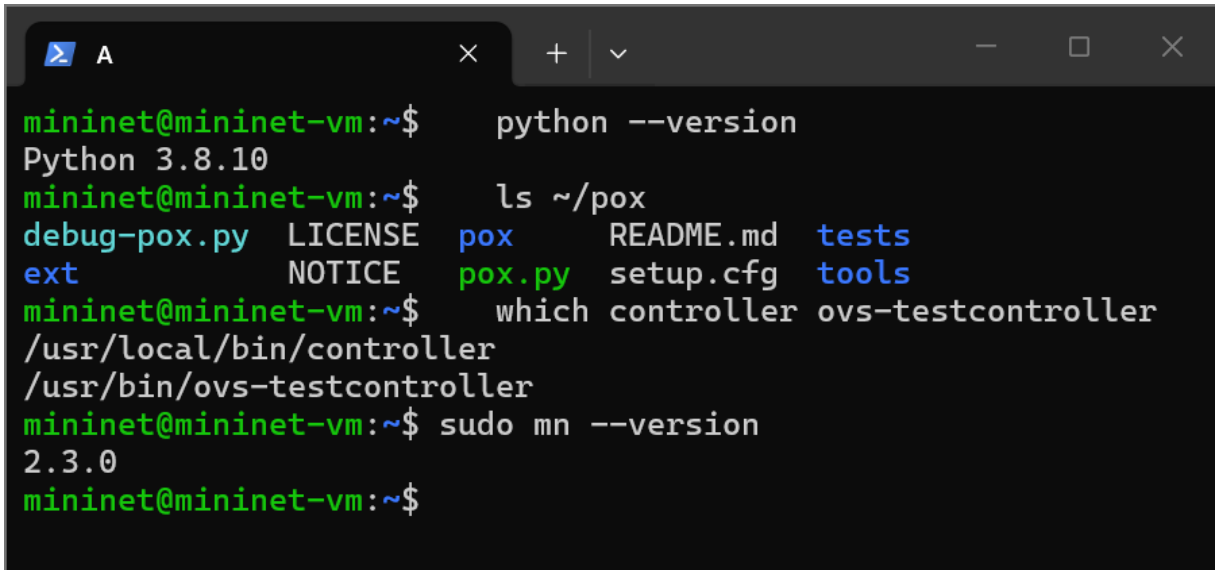


Figure 1 – Vérification de l'environnement : Python, dossier POX, contrôleurs et Mininet.

Les scripts du sujet utilisent `environ['HOME']` pour retrouver le dossier de POX. Sous `sudo`, `HOME` devient `/root` et POX n'est plus trouvé. Je lance donc tous les scripts avec `sudo -E python3 ...`, qui conserve `HOME`. Avant chaque lancement, je nettoie aussi les processus restants avec `sudo pkill -f pox.py` et `sudo mn -c`.

3 Exercice 1 – Premier déploiement

Mise en place. Dans une première fenêtre, je lance POX avec le composant d'apprentissage de couche 2 :

```
cd ~/pox
sudo python3 pox.py openflow.of_01 --port=6633 forwarding.l2_learning
```

POX affiche `POX 0.7.0 (gar) is up` (figure 2) et reste au premier plan pendant tout le test. Dans une seconde fenêtre, je lance Mininet avec la topologie par défaut, reliée à ce contrôleur (le `sudo min` du sujet est une faute de frappe pour `mn`) :

```
sudo mn --controller=remote,port=6633
```

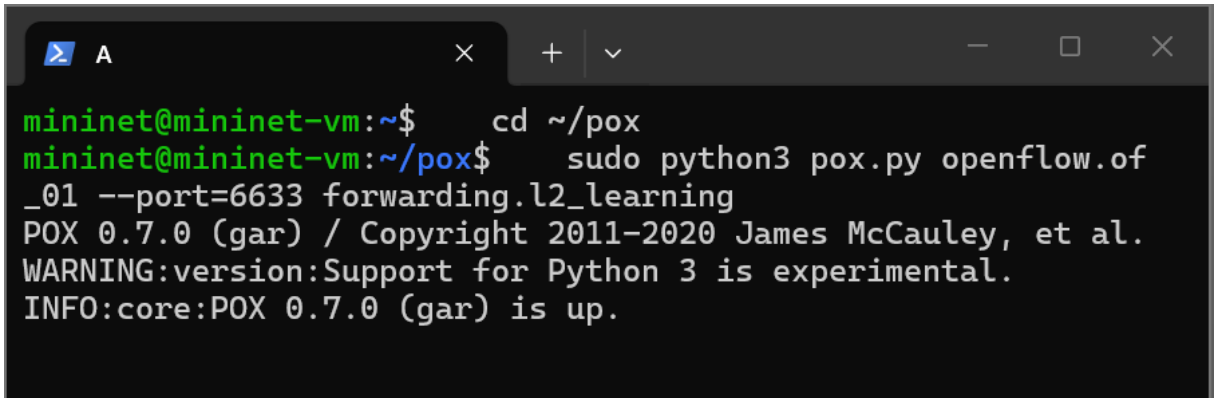


Figure 2 – POX démarré avec `forwarding.l2_learning` sur le port 6633.

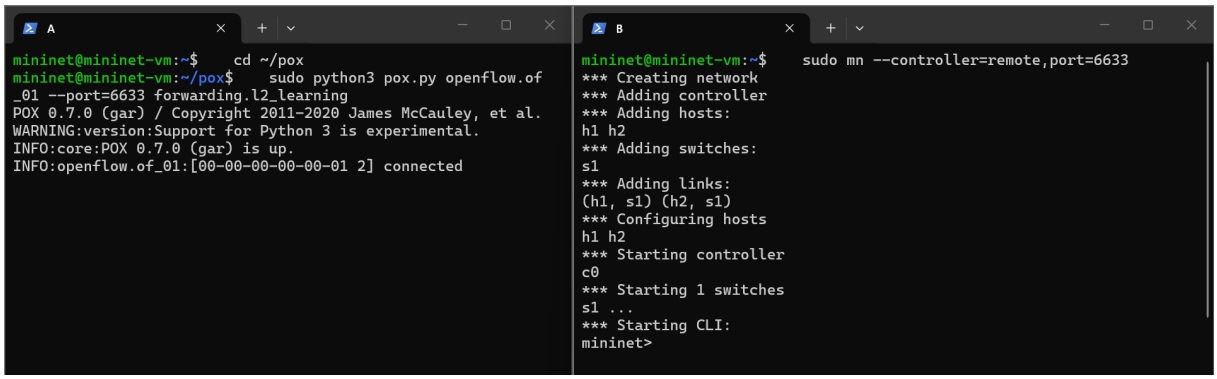


Figure 3 – Mininet démarré (à droite) et connexion du switch visible dans le journal de POX.

Réponse

Comment je vérifie que tout fonctionne ? Trois choses :

1. *Le démarrage* : Mininet arrive à son invite sans erreur de connexion au contrôleur.
2. *La connexion du switch* : le journal de POX affiche une ligne `[00-00-00-00-00-01 1] connected` dès que Mininet démarre (figure 3).
3. *Le comportement du switch* : `pingall` ne perd aucun paquet, le premier ping est plus lent que les suivants (le switch passe par le contrôleur), et la table de flux contient des flux précis posés par POX.

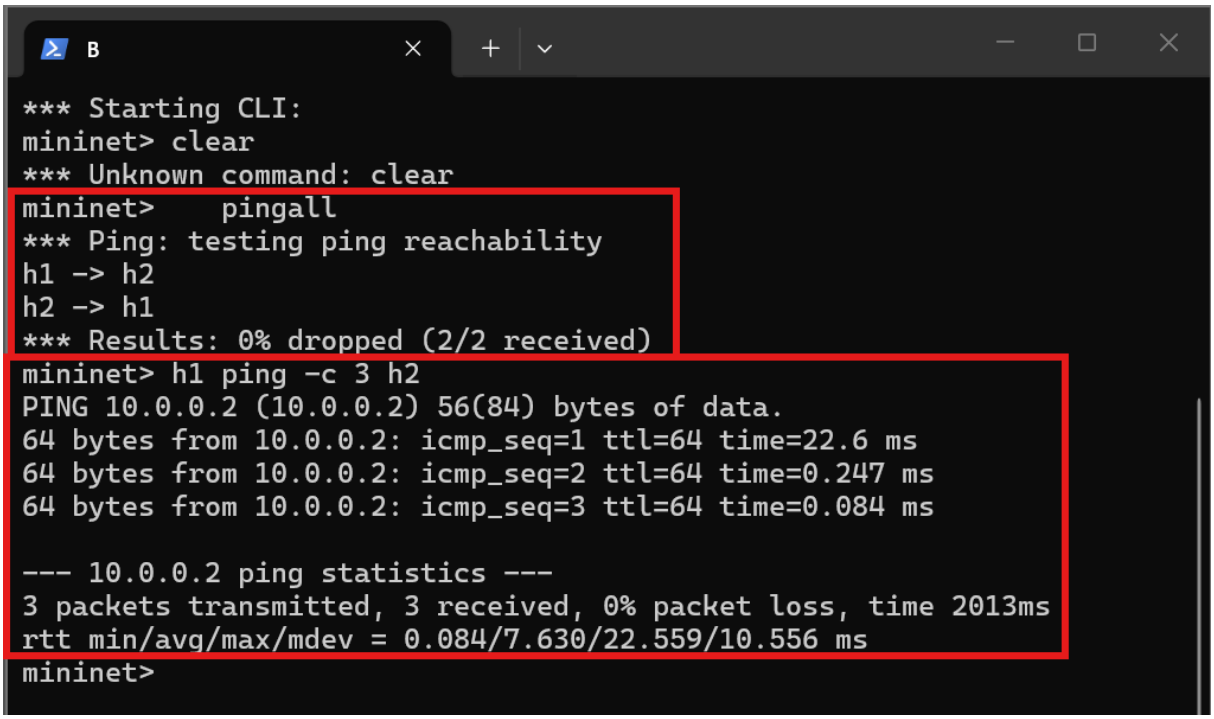


Figure 4 – `pingall` et ping entre h1 et h2 : le premier échange est plus lent.

La table de flux de `s1` (figure 5) contient les flux installés par `l2_learning`. Ils filtrent sur le port d'entrée et sur les adresses source et destination, et d'après le code de POX ils expirent après 10 s sans trafic (30 s au maximum). POX inonde d'abord le premier paquet, apprend l'adresse source et le port, puis pose un flux : les paquets suivants ne passent plus par le contrôleur.

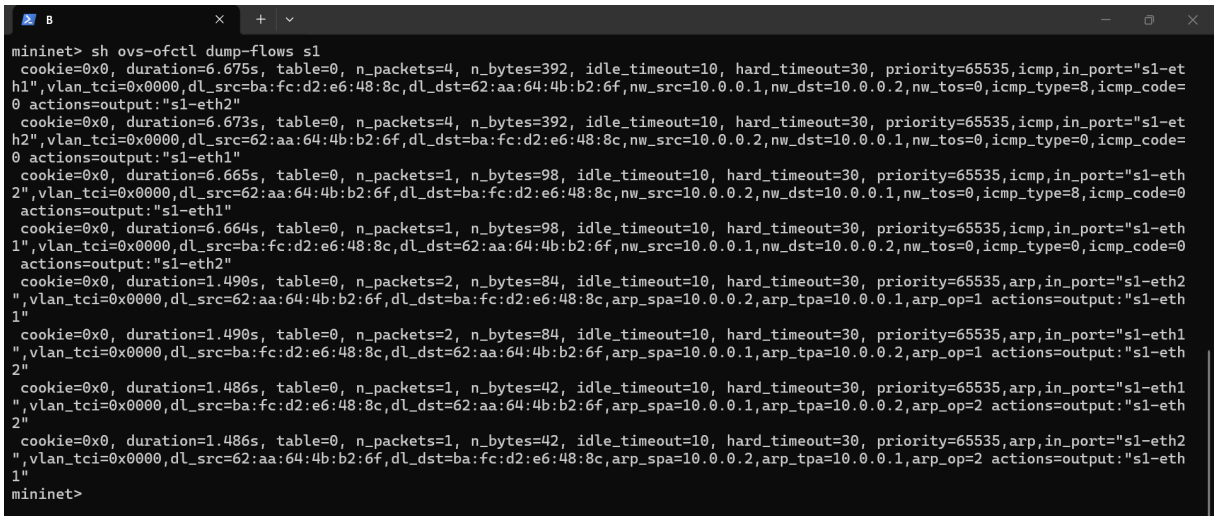


Figure 5 – Flux installés par `l2_learning` dans le switch s1.

4 Exercice 2 – Passerelle avec POX

4.1 Les deux parties du script

Réponse

Première partie : elle définit deux classes. `CustomPOX` lance POX dans le dossier `~/pox` avec le composant à charger (paramètre `cargs`), et `CustomSwitch` connecte chaque switch au contrôleur indiqué dans le dictionnaire `cmap`.
Seconde partie : elle construit le réseau. Elle crée les deux contrôleurs (POX et le contrôleur Mininet), les deux switches, les deux hôtes et les liens, puis démarre l'ensemble, lance un `pingAll` et ouvre la ligne de commande. Elle associe s1 à POX et s2 au contrôleur natif.

4.2 Exécution du script d'origine

J'ai lancé `passerelle.py` tel quel : `pingAll` donne 100 % de pertes (figure 6).

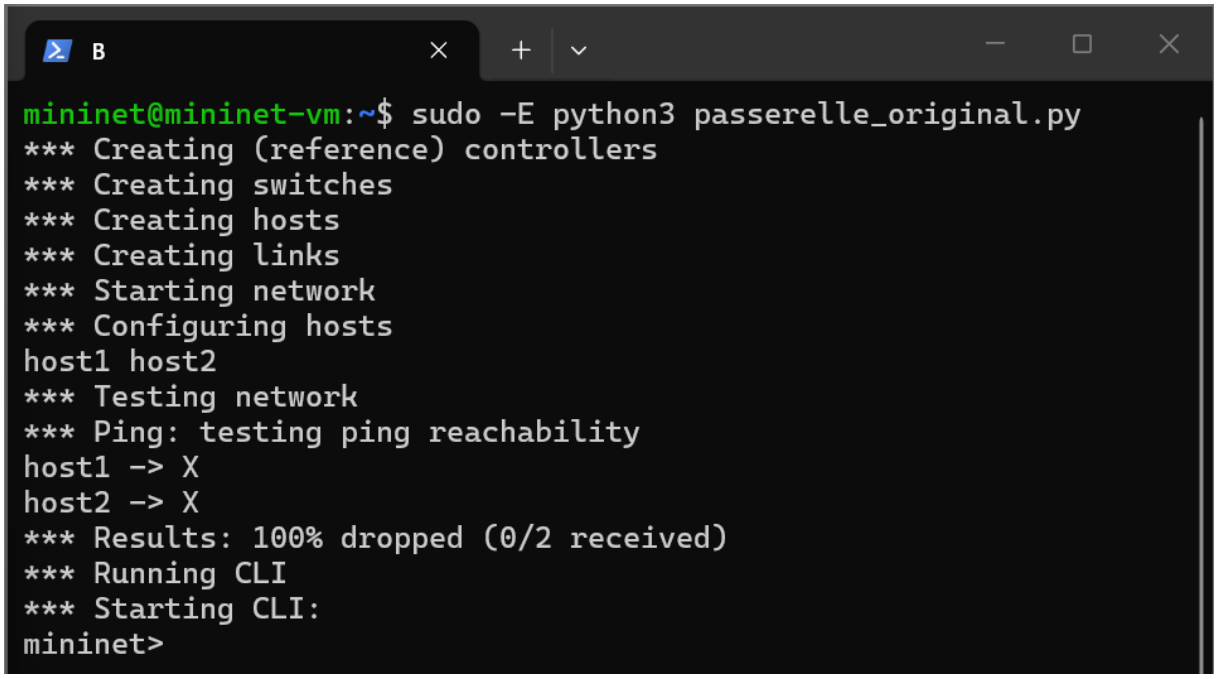


Figure 6 – Exécution du `passerelle.py` d'origine : `pingAll` à 100 % de pertes.

En regardant les contrôleurs (figure 7), le port 6633 est tenu par le processus `controller`, c'est-à-dire le contrôleur natif de Mininet, alors que le journal de POX dit `Error 98 while binding 0.0.0.0:6633: Address already in use`. Dans le script d'origine, les deux contrôleurs sont configurés sur le même port : un seul peut démarrer et POX n'est jamais en écoute. Cela ne change pas la réponse à la question (voir ci-dessous), mais ma version modifiée donne le port 6634 au contrôleur natif.

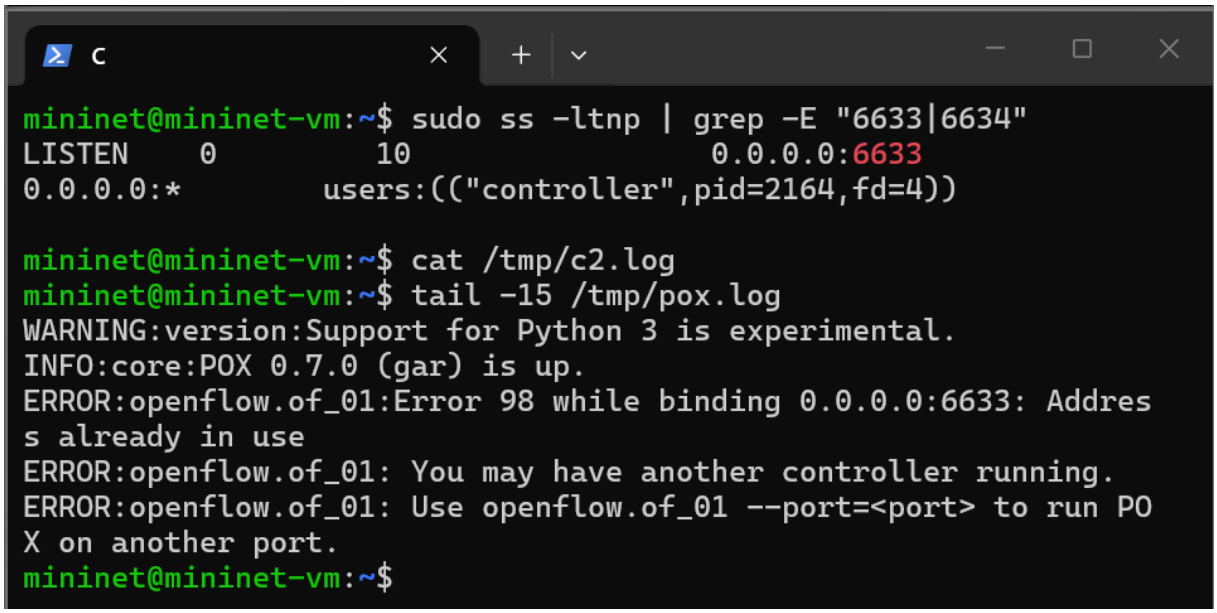


Figure 7 – Port 6633 pris par le contrôleur natif, journal d'erreur de POX et journal de c2.

Réponse

Le ping fonctionne-t-il ? Non.
Pourquoi ? Les deux hôtes sont dans des sous-réseaux différents (`10.0.10.0/24` et `10.0.30.0/24`) et n'ont aucune route pour joindre l'autre : le ping échoue dès l'hôte avec `Network is unreachable`. Chaque hôte n'a qu'une seule route, vers son propre sous-réseau (figure 8). De plus, les switches travaillent en couche 2 et ne routent pas.

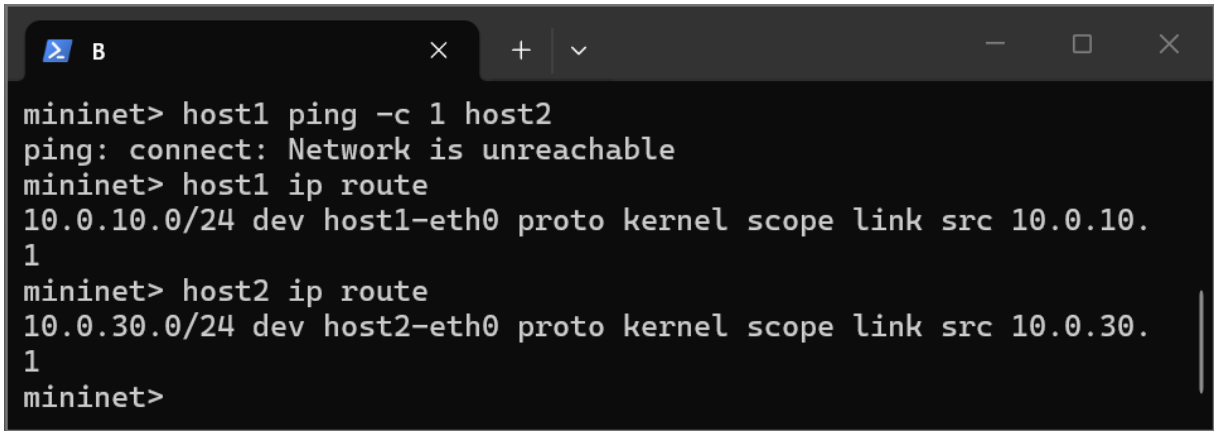


Figure 8 – Ping en échec et tables de routage de host1 et host2 : une seule route chacun.

4.3 POX comme passerelle

Réponse

Que doit faire POX pour résoudre le problème ? Se comporter comme un routeur de couche 3 : répondre aux requêtes ARP pour les adresses de passerelle avec sa propre adresse MAC, puis réécrire l'adresse MAC de destination et transmettre le paquet vers le bon port. Le composant `forwarding.l3_learning` sait le faire avec l'option `-fakeways`, qui déclare les adresses de passerelle.

J'ai modifié `passerelle.py` pour que POX charge ce composant, avec les deux adresses de passerelle (une par sous-réseau) :

```
cargs='openflow.of_01 --port=%s forwarding.l3_learning --fakeways
    =10.0.10.254,10.0.30.254'
```

Le contrôleur natif passe sur le port 6634 pour qu'il n'y ait plus de conflit. Après relance, le ping échoue toujours (figure 9).

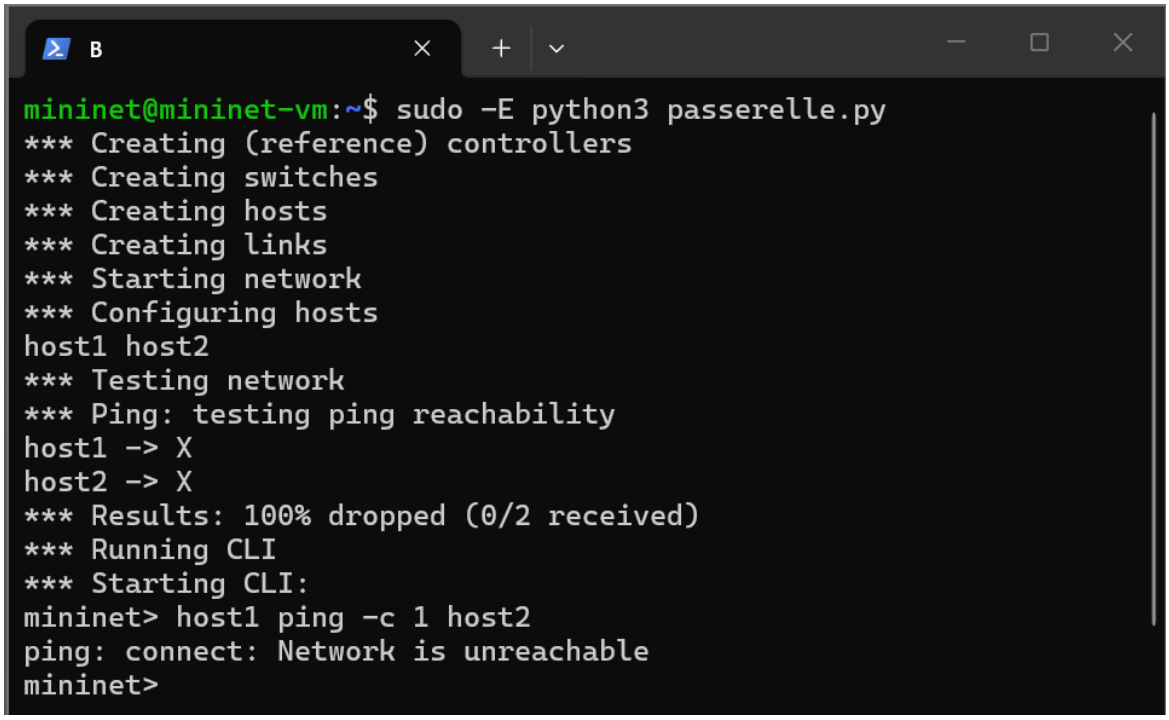


Figure 9 – Script modifié : `pingAll` et ping toujours en échec.

Réponse

Pourquoi cela ne fonctionne-t-il toujours pas ? POX est prêt à jouer le rôle de passerelle, mais les hôtes ne savent pas qu'elle existe : ils n'ont toujours pas de route pour joindre l'autre sous-réseau.

Que changer dans la configuration de h1 et h2 ? Ajouter à chacun une route par défaut qui passe par l'adresse de passerelle de son sous-réseau. Depuis la ligne de commande de Mininet :

```
host1 ip route add default via 10.0.10.254
host2 ip route add default via 10.0.30.254
```

Les routes par défaut apparaissent dans la table de routage de l'hôte (figure 10).

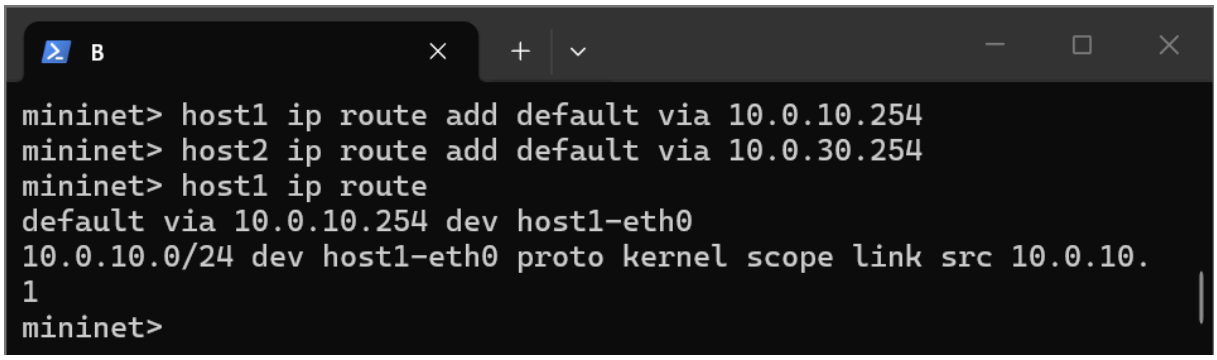


Figure 10 – Ajout des routes par défaut et table de routage de host1.

Le ping fonctionne ensuite entre les deux hôtes (figure 11). Les deux premiers paquets d'un nouvel échange sont perdus : `l3_learning` pose un flux pour le paquet qui le déclenche mais ne le retransmet pas (Open vSwitch ne le met pas en mémoire tampon), et il faut un flux par sens. Ces flux expirent après 10 s d'inactivité d'après le code de POX, et un `pingall`, qui n'envoie qu'un ping par sens, échoue donc s'il arrive après une pause. Pour garder les flux actifs, j'ai lancé deux pings continus en arrière-plan, un dans chaque sens, avant de lancer `pingall`.

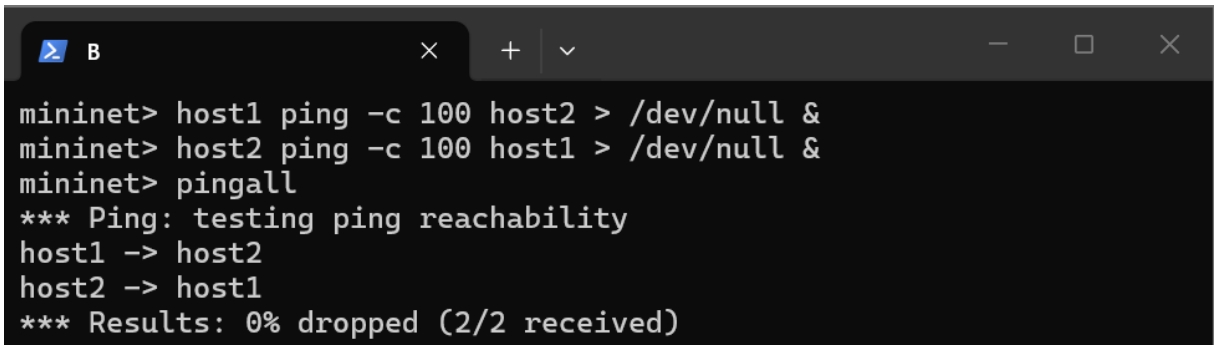


Figure 11 – Pings en arrière-plan dans les deux sens et `pingall` sans perte.

La table de flux de `s1` (figure 12) montre la passerelle à l'œuvre. Les hôtes envoient leurs paquets à l'adresse MAC de la passerelle (`00:00:00:00:00:01`, dérivée de l'identifiant du switch), et chaque flux réécrit l'adresse de destination (`mod_dl_dst`) avant de transmettre le paquet. Il y a quatre flux : requête et réponse ICMP, dans chaque sens, chacun avec son `icmp_type` (8 et 0), et un `idle_timeout` de 10 s.

```
mininet> sh ovs-ofctl dump-flows s1
 cookie=0x0, duration=24.996s, table=0, n_packets=25, n_bytes=2450
, idle_timeout=10, priority=65535,icmp,in_port="s1-eth1",vlan_tci=
0x0000,dl_src=00:00:00:00:10:01,dl_dst=00:00:00:00:00:01,nw_src=10
.0.10.1,nw_dst=10.0.30.1,nw_tos=0,icmp_type=8,icmp_code=0 actions=
mod_dl_dst:00:00:00:00:20:01,output:"s1-eth2"
 cookie=0x0, duration=23.969s, table=0, n_packets=24, n_bytes=2352
, idle_timeout=10, priority=65535,icmp,in_port="s1-eth2",vlan_tci=
0x0000,dl_src=00:00:00:00:20:01,dl_dst=00:00:00:00:00:01,nw_src=10
.0.30.1,nw_dst=10.0.10.1,nw_tos=0,icmp_type=0,icmp_code=0 actions=
mod_dl_dst:00:00:00:00:10:01,output:"s1-eth1"
 cookie=0x0, duration=15.105s, table=0, n_packets=15, n_bytes=1470
, idle_timeout=10, priority=65535,icmp,in_port="s1-eth2",vlan_tci=
0x0000,dl_src=00:00:00:00:20:01,dl_dst=00:00:00:00:00:01,nw_src=10
.0.30.1,nw_dst=10.0.10.1,nw_tos=0,icmp_type=8,icmp_code=0 actions=
mod_dl_dst:00:00:00:00:10:01,output:"s1-eth1"
 cookie=0x0, duration=14.085s, table=0, n_packets=14, n_bytes=1372
, idle_timeout=10, priority=65535,icmp,in_port="s1-eth1",vlan_tci=
0x0000,dl_src=00:00:00:00:10:01,dl_dst=00:00:00:00:00:01,nw_src=10
.0.10.1,nw_dst=10.0.30.1,nw_tos=0,icmp_type=0,icmp_code=0 actions=
mod_dl_dst:00:00:00:00:20:01,output:"s1-eth2"
mininet>
```

Figure 12 – Flux de s1 : réécriture de l'adresse MAC de destination (`mod_dl_dst`).

5 Exercice 3 – Routage multi-chemins

5.1 La topologie

Le script `diamond.py` construit quatre switches en losange (un anneau s1–s2–s3–s4–s1), avec un hôte sur chaque switch. Les adresses MAC et IP sont fixes. Chaque hôte est dans son propre sous-réseau.

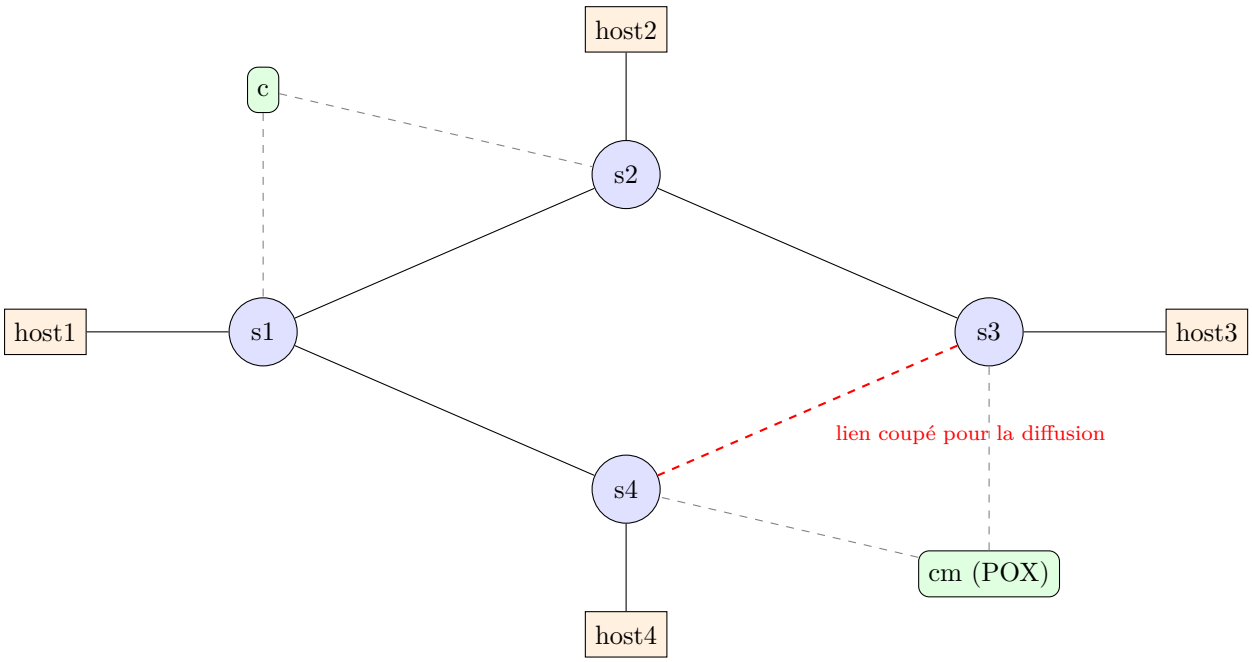


Figure 13 – Topologie de `diamond.py`. Le contrôleur `c` pilote s1 et s2, le contrôleur POX `cm` (composant `misc.tp`) pilote s3 et s4. Les traits pointillés gris sont les connexions OpenFlow.

Switch	eth1	eth2	eth3
s1	host1 (10.0.10.1)	s4	s2
s2	host2 (10.0.20.1)	s3	s1
s3	host3 (10.0.30.1)	s2	s4
s4	host4 (10.0.40.1)	s1	s3

Table 1 – Voisin joint par chaque port, d'après les noms de ports de `diamond.py`.

Les adresses MAC des hôtes sont `00:00:00:00:10:01` (host1), `00:00:00:00:20:01` (host2), `00:00:00:00:30:01` (host3) et `00:00:00:00:40:01` (host4).

Les numéros de port OpenFlow ne correspondent pas aux noms. Mininet numérote les ports dans l'ordre de création des liens, alors que `diamond.py` impose les noms : par exemple `s1-eth3` est le deuxième port de s1 (figure 14). Ma solution retrouve donc les ports par leur nom et jamais par leur numéro.

```
c
mininet@mininet-vm:~$ sudo ovs-vsctl list-ports s3
s3-eth1
s3-eth2
s3-eth3
mininet@mininet-vm:~$ for s in s1 s2 s3 s4; do sudo ovs-ofctl show $s | grep -E "
^ [0-9]+\("; done
 1(s1-eth1): addr:00:00:00:00:10:02
 2(s1-eth2): addr:00:00:00:00:20:01
 3(s1-eth3): addr:00:00:00:00:30:01
 1(s2-eth1): addr:00:00:00:00:10:02
 2(s2-eth2): addr:00:00:00:00:20:01
 3(s2-eth3): addr:00:00:00:00:30:01
 1(s3-eth1): addr:00:00:00:00:10:02
 2(s3-eth2): addr:00:00:00:00:20:01
 3(s3-eth3): addr:00:00:00:00:30:01
 1(s4-eth1): addr:00:00:00:00:10:02
 2(s4-eth2): addr:00:00:00:00:20:01
 3(s4-eth3): addr:00:00:00:00:30:01
mininet@mininet-vm:~$
```

Figure 14 – Noms et numéros OpenFlow des ports de s1 à s4.

5.2 Fonctionnement de `tp.py` (version d'origine)

Fonction	Rôle
`launch()`	Point d'entrée du composant. Elle s'abonne à l'événement `ConnectionUp` : chaque fois qu'un switch se connecte, elle crée un objet `Diamond` pour lui.
`__init__(connection)`	Garde la connexion avec le switch, s'abonne à ses événements (`addListeners`) et crée la table `mac_to_port`.
`_handle_PacketIn(event)`	Appelée pour chaque paquet que le switch envoie au contrôleur. Elle vérifie que le paquet est complet, puis appelle le comportement choisi (hub ou `act_less_stupidly`).
`act_like_hub(...)`	Renvoie le paquet sur tous les ports sauf celui d'entrée.
`act_less_stupidly(...)`	Vide dans le fichier fourni : c'est l'emplacement de la solution.
`resend_packet(...)`	Envoie au switch un `packet_out` qui fait ressortir le paquet par le port demandé.

5.3 Le problème observé

J'ai lancé le réseau avec la version d'origine de `tp.py`, où tous les switches POX se comportent en hub, puis lancé `tcpdump` sur `s1-eth1` (limité à quelques paquets) avant un ping entre host1 et host2.

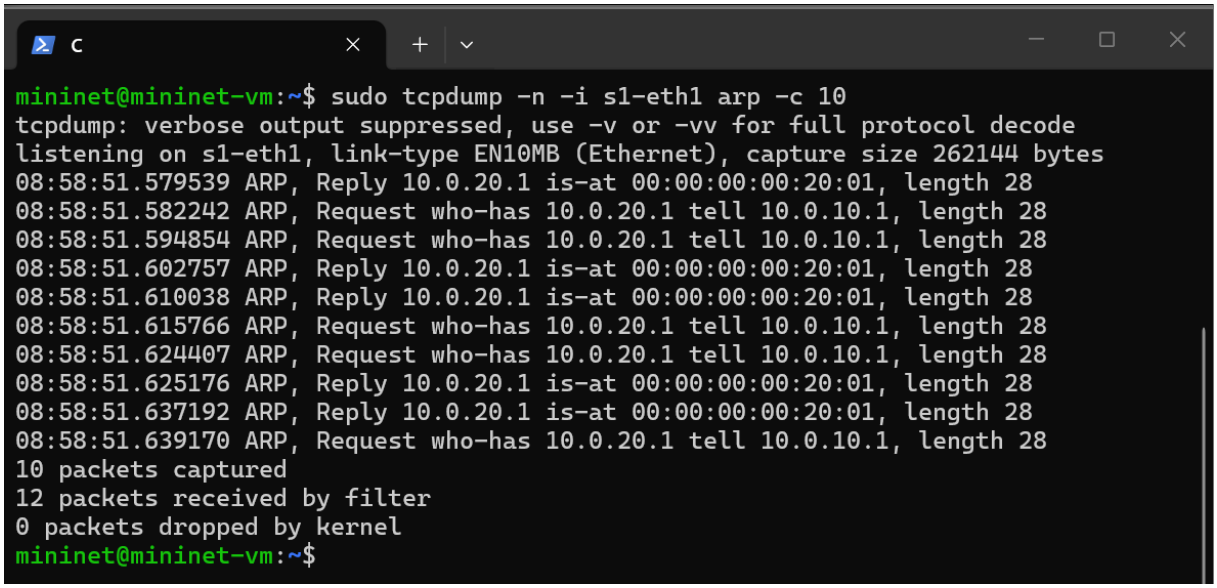


Figure 15 – `tcpdump` sur `s1-eth1` avec le comportement de hub : la même requête ARP se répète.

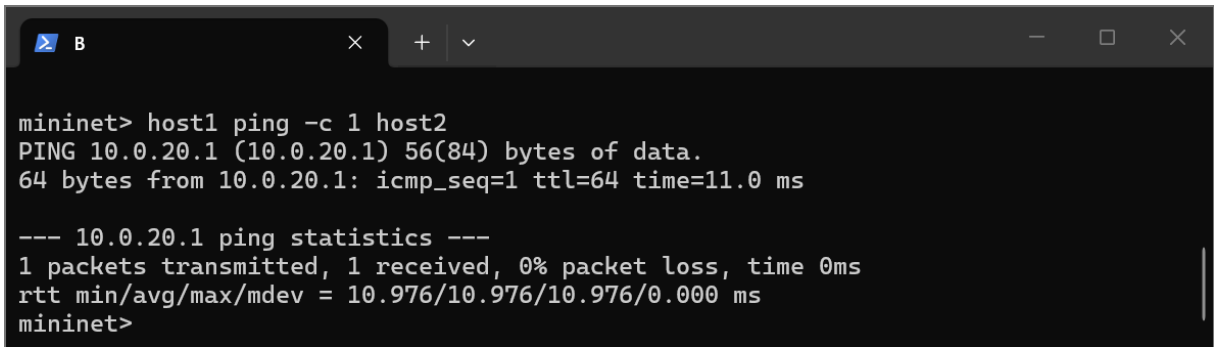


Figure 16 – Ping entre host1 et host2 avec le comportement de hub.

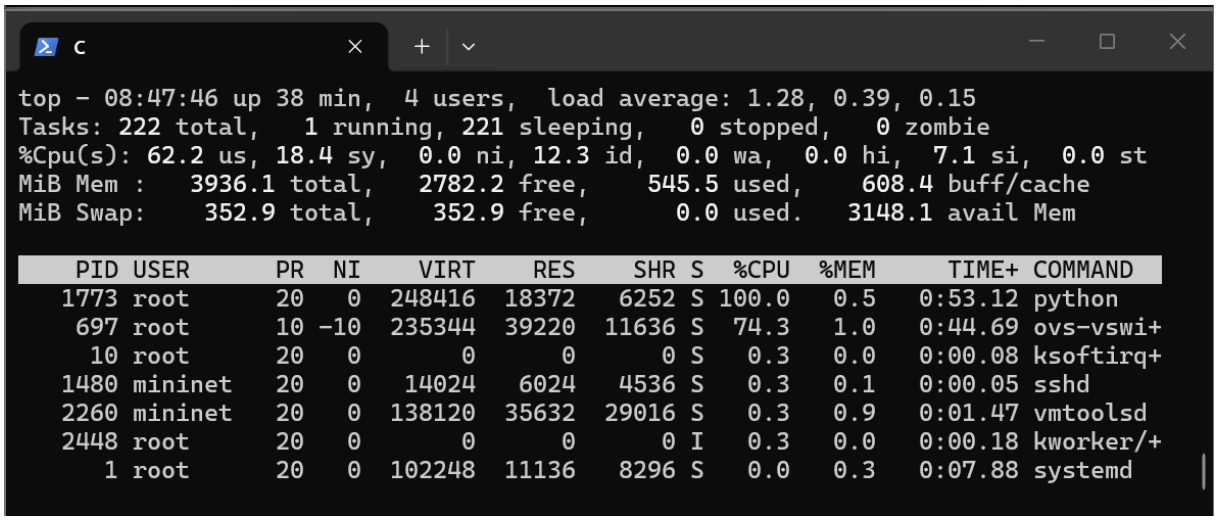


Figure 17 – Charge CPU élevée pendant la tempête.

Réponse

Que s'est-il passé ? La même requête ARP revient sans cesse sur `s1-eth1` (figure 15), le ping ne se déroule pas normalement (figure 16) et la charge CPU monte (figure 17).
Le comportement du réseau est-il correct ? Non.
Quel est le problème ? Les quatre switches forment une boucle (s1-s2-s3-s4-s1) et les switches pilotés par `tp.py` se comportent en hub : chacun renvoie toute trame sur tous ses ports sauf celui d'entrée. Une trame de diffusion fait donc le tour de l'anneau indéfiniment, puisqu'une trame Ethernet n'a pas de TTL, et chaque passage la duplique. C'est une *tempête de diffusion*. Elle sature aussi le contrôleur, car chaque trame remonte en `packet_in`.

5.4 La solution

Principe. L'idée est d'ouvrir la boucle uniquement pour la diffusion, sans perdre le lien s3–s4 pour le trafic normal.

— **Diffusion.** Les trames de diffusion, de multidiffusion et celles à destination d'une adresse MAC inconnue suivent un arbre. Un hôte envoie vers le haut (s3 vers s2, s4 vers s1), le switch du haut diffuse, et un switch qui reçoit par le haut ne livre qu'à son hôte. Le lien s3–s4 ne transporte jamais de diffusion : la boucle est ouverte.
— **Unicast.** Quand le destinataire est connu, le paquet prend le chemin le plus court. Vers l'hôte d'un switch voisin, il passe directement par le lien qui les relie (donc s3–s4 sert au trafic host3–host4). Vers l'hôte du switch opposé, il remonte par l'arbre.
— **Aucune retransmission en chaîne.** Un paquet unicast qui arrive d'un autre switch est livré s'il est destiné à l'hôte local, sinon il est supprimé. Il ne peut donc ni tourner ni être dupliqué.

Arrivée par	Destination	Action
port de l'hôte	diffusion, multidiffusion ou MAC inconnue	vers le haut
port de l'hôte	hôte du switch voisin	lien direct
port de l'hôte	hôte du switch opposé	vers le haut
lien du haut	diffusion, multidiffusion ou MAC inconnue	livrée à l'hôte local
lien du haut ou lien coupé	hôte local	livrée à l'hôte local
lien du haut ou lien coupé	autre hôte	supprimée
lien coupé	diffusion ou multidiffusion	supprimée

Table 2 – Règles appliquées par s3 et s4 (« haut » : s2 pour s3, s1 pour s4 ; « lien coupé » : lien s3–s4).

Chemins obtenus. Les switches s1 et s2 sont pilotés par le contrôleur natif (apprentissage par inondation), qui s'accorde avec cet arbre : host1–host2 par s1–s2, host2–host3 par s2–s3, host3–host4 par s3–s4, host4–host1 par s4–s1, host1–host3 par s2 et host2–host4 par s1. Les quatre liens entre switches sont donc utilisés.

Implémentation (code complet en annexe A).

- Au démarrage de chaque switch, `__init__` lit la liste de ses ports dans la réponse `features` et construit un dictionnaire nom du port → numéro OpenFlow. Les constantes `VOISINS` et `LIEN_HAUT` décrivent la topologie, et `HOTES` associe l'adresse MAC de chaque hôte à son switch.
- `act_less_stupidly` applique le tableau des règles. Pour les paquets unicast et les trames de multidiffusion, `installer_flux` pose un flux `(port d'entrée, MAC destination)` avec un délai d'inactivité de 10 s et un délai maximal de 30 s : les paquets suivants ne passent plus par le contrôleur. Un flux sans action supprime les paquets que la règle interdit.
- Le paquet qui a déclenché la décision est renvoyé par `resend_packet`. Je n'utilise pas le tampon du flux pour cela, car Open vSwitch ne met pas ces paquets en mémoire.
- Un switch absent de `LIEN_HAUT` (s1 ou s2 s'ils étaient pilotés par POX) se comporte en commutateur à apprentissage classique (`apprendre_et_inonder`), pour que le composant reste utilisable sur les quatre switches.

5.5 Tests

J'ai relancé le réseau avec la nouvelle version de `tp.py`. Le journal de POX confirme que s3 et s4 ont trouvé leurs ports (figure 18).

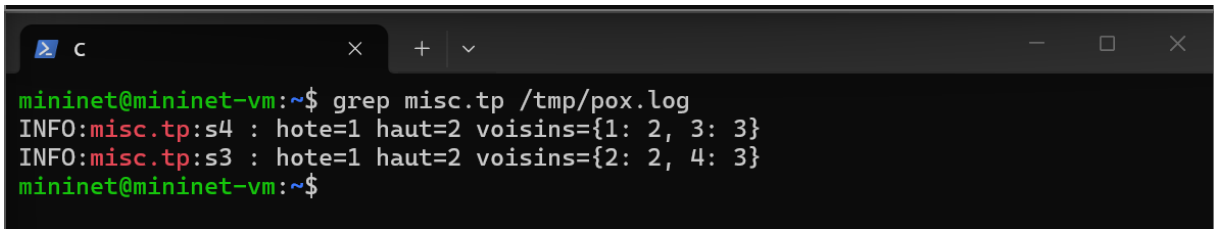


Figure 18 – Journal de POX : ports d'hôte, de remontée et de voisins retrouvés par nom pour s3 et s4.

Communication entre tous les hôtes. `pingall` ne perd aucun paquet (figure 19).

```
mininet> pingall
*** Ping: testing ping reachability
host1 -> host2 host3 host4
host2 -> host1 host3 host4
host3 -> host1 host2 host4
host4 -> host1 host2 host3
*** Results: 0% dropped (12/12 received)
mininet>
```

Figure 19 – `pingall` sans perte avec la solution.

Pas de boucle ni de duplication. Après un redémarrage, `tcpdump` sur `s1-eth1` ne montre qu'une requête ARP, sa réponse, puis l'échange ICMP, sans répétition (figure 20). Les pings entre host1 et host3, host2 et host4, host3 et host4 ne contiennent aucun paquet dupliqué (`DUP!`, figure 21).

```
c
mininet@mininet-vm:~$ sudo tcpdump -XX -n -i s1-eth1 -c 4 'arp or icmp'
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on s1-eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
09:09:19.819636 ARP, Request who-has 10.0.20.1 tell 10.0.10.1, length 28
        0x0000:  ffff ffff ffff 0000 0000 1001 0806 0001  ................
        0x0010:  0800 0604 0001 0000 0000 1001 0a00 0a01  ................
        0x0020:  0000 0000 0000 0a00 1401                 ..........
09:09:19.824106 ARP, Reply 10.0.20.1 is-at 00:00:00:00:20:01, length 28
        0x0000:  0000 0000 1001 0000 0000 2001 0806 0001  ................
        0x0010:  0800 0604 0002 0000 0000 2001 0a00 1401  ................
        0x0020:  0000 0000 1001 0a00 0a01                 ..........
09:09:19.824161 IP 10.0.10.1 > 10.0.20.1: ICMP echo request, id 4544, seq 1, length 64
        0x0000:  0000 0000 2001 0000 0000 1001 0800 4500  ............E.
        0x0010:  0054 c1fb 4000 4001 46ac 0a00 0a01 0a00  .T..@.@.F.......
        0x0020:  1401 0800 76a7 11c0 0001 2fd8 dc6a 0000  ....v...../..j..
        0x0030:  0000 9881 0c00 0000 0000 1011 1213 1415  ................
        0x0040:  1617 1819 1a1b 1c1d 1e1f 2021 2223 2425  ..........!"#$%
        0x0050:  2627 2829 2a2b 2c2d 2e2f 3031 3233 3435  &'()*+,-./012345
        0x0060:  3637                                     67
09:09:19.825841 IP 10.0.20.1 > 10.0.10.1: ICMP echo reply, id 4544, seq 1, length 64
        0x0000:  0000 0000 1001 0000 0000 2001 0800 4500  ............E.
        0x0010:  0054 4ce6 0000 4001 fbc1 0a00 1401 0a00  .TL...@.........
        0x0020:  0a01 0000 7ea7 11c0 0001 2fd8 dc6a 0000  ....~...../..j..
        0x0030:  0000 9881 0c00 0000 0000 1011 1213 1415  ................
        0x0040:  1617 1819 1a1b 1c1d 1e1f 2021 2223 2425  ..........!"#$%
        0x0050:  2627 2829 2a2b 2c2d 2e2f 3031 3233 3435  &'()*+,-./012345
        0x0060:  3637                                     67
4 packets captured
4 packets received by filter
0 packets dropped by kernel
mininet@mininet-vm:~$
```

Figure 20 – `tcpdump` avec la solution : une requête ARP, une réponse, puis l'ICMP.

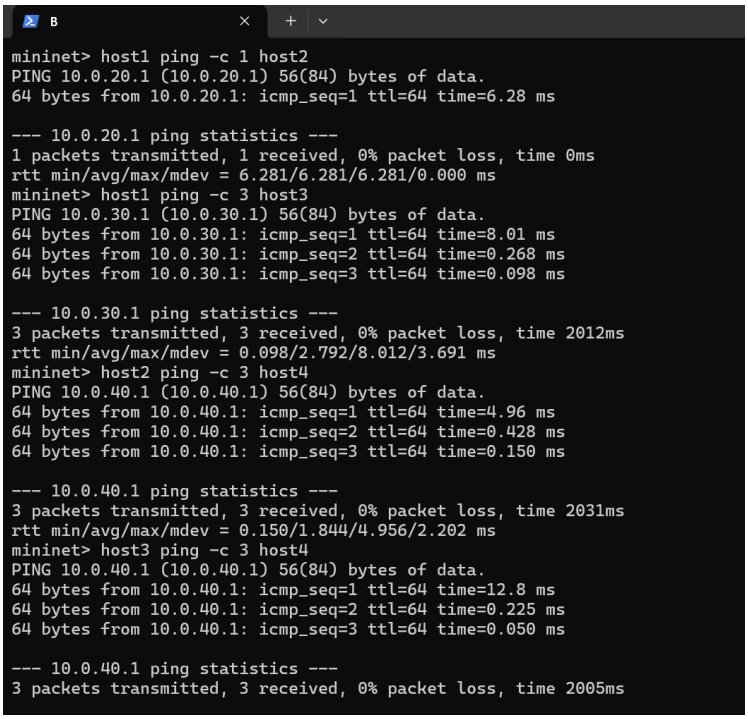


Figure 21 – Pings entre hôtes éloignés, sans paquet dupliqué.

Flux installés. Les tables de s3 et s4 contiennent les flux posés par le composant, avec leurs délais (figure 22).



Figure 22 – Flux installés sur s3 et s4 par `tp.py`.

Utilisation des liens. Après un `pingall` sur un réseau fraîchement démarré, les compteurs de paquets de toutes les interfaces entre switches sont non nuls : `s1-eth2`, `s1-eth3`, `s2-eth2`, `s2-eth3`, `s3-eth2`, `s3-eth3`, `s4-eth2` et `s4-eth3` (figure 23). Le lien s3–s4, absent des diffusions, est utilisé par le trafic host3–host4.

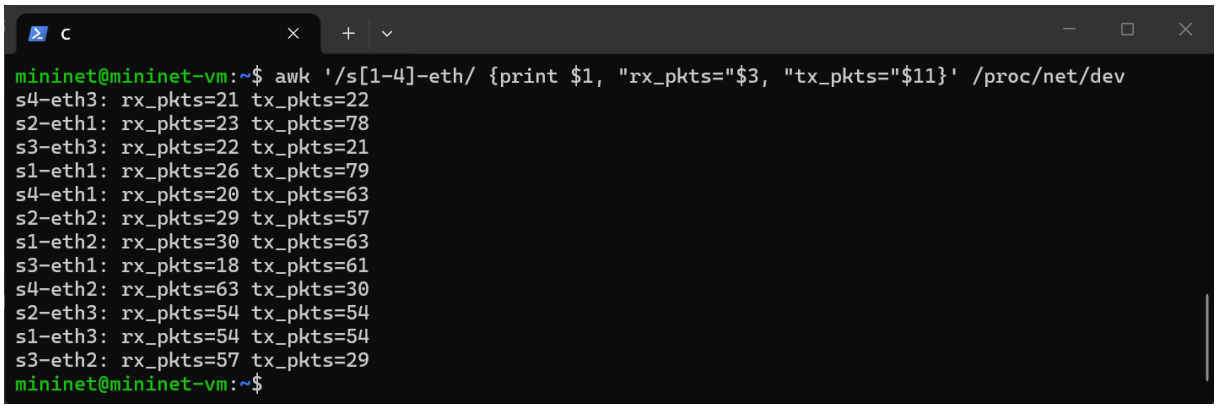


Figure 23 – Compteurs de paquets des interfaces des switches après `pingall`.

Réponse

Les trois critères du sujet sont remplis : tous les hôtes communiquent, les paquets ne sont ni dupliqués ni mis en circulation infinie, et tous les liens entre switches sont utilisés.

5.6 Limites

La solution est statique : elle connaît la topologie et les adresses MAC des hôtes (niveau « de base » des pistes du sujet), et les chemins ne s'adaptent pas à la charge. Elle ne réagit pas non plus à une panne de lien, puisque s1 et s2 sont pilotés par un autre contrôleur et que les messages d'état de port ne sont pas traités. Pour aller plus loin, on pourrait découvrir la topologie avec LLDP, répartir les flux par une fonction de hachage entre plusieurs chemins de même coût, et recalculer les chemins quand un port change d'état.

6 Modifications des fichiers fournis

J'ai modifié `tp.py` et `passerelle.py` comme demandé. Pour `diamond.py`, le sujet précise qu'il ne doit pas être modifié. Je n'ai touché que la partie marquée comme modifiable, et uniquement pour les raisons ci-dessous (détail des changements en annexe B).

Fichier	Modification	Raison
`diamond.py`	`cargs` de POX : composant `misc.tp`, passé en paramètre	Le fichier fourni charge `forwarding.l3_learning` alors que le sujet dit que `cm` charge `misc.tp`
`diamond.py`	`cmap` : s1 et s2 sur `c`, s3 et s4 sur `cm`	Conforme au texte du sujet (deux switches par contrôleur)
`diamond.py`	Contrôleur `c` sur le port 6634	Les deux contrôleurs étaient sur 6633 (un seul peut démarrer)
`diamond.py`	Routes vers les autres sous-réseaux sur chaque hôte	Le fichier ajoute `dev eth0`, interface qui n'existe pas (elle s'appelle `host1-eth0`)
`diamond.py`	`net.pingAll()` supprimé	Éviter de déclencher la tempête avant d'ouvrir `tcpdump`
`tcpdump`		
`passerelle.py`	Composant `l3_learning` avec `--fakeways`	Demandé par le sujet
`passerelle.py`	Contrôleur natif sur le port 6634	Conflit de port avec POX (S07)
`tp.py`	Fonction `act_less_stupidly` et fonctions associées	Solution de l'exercice 3

Table 3 – Modifications apportées et raisons.

Les noms des hôtes dans Mininet sont `host1` à `host4`, et non `h1` à `h4` comme dans le sujet : les commandes sont adaptées en conséquence.

7 Bilan

Exercice	Contenu	Réussi	Captures
1	Prise en main de POX avec `l2_learning`	Oui	S02 à S05
2	Passerelle avec `l3_learning`	Oui	S06 à S12
3	Routage multi-chemins avec `tp.py`	Oui	S13 à S22

Ce TP montre qu'un contrôleur peut transformer un switch en hub, en commutateur ou en routeur simplement par le code qu'on lui donne. La difficulté de l'exercice 3 est de ne pas casser ce que fait le reste du réseau : il faut tenir compte des switches qu'on ne contrôle pas, des numéros de port qui ne suivent pas les noms et de l'absence de TTL au niveau Ethernet.

A Code de `tp.py`

```
"""
This component acts as a simple hub, but can be modified to act like an L2
learning switch.

"""

from pox.core import core
import pox.openflow.libopenflow_01 as of

log = core.getLogger()
```

```
VOISINS = {
  1: {2: 'eth3', 4: 'eth2'},
  2: {1: 'eth3', 3: 'eth2'},
  3: {2: 'eth2', 4: 'eth3'},
  4: {1: 'eth2', 3: 'eth3'},
}

LIEN_HAUT = {3: 2, 4: 1}

HOTES = {
  '00:00:00:00:10:01': 1,
  '00:00:00:00:20:01': 2,
  '00:00:00:00:30:01': 3,
  '00:00:00:00:40:01': 4,
}

IDLE_TIMEOUT = 10
HARD_TIMEOUT = 30


class Diamond (object):
  """
  A Diamond object is created for each switch that connects.
  A Connection object for that switch is passed to the __init__ function.
  """
  def __init__ (self, connection):
    # Keep track of the connection to the switch so that we can
    # send it messages!
    self.connection = connection

    # This binds our PacketIn event listener
    connection.addListeners(self)

    # Use this table to keep track of which ethernet address is on
    # which switch port (keys are MACs, values are ports).
    self.mac_to_port = {}

    self.dpid = connection.dpid
    self.numero = {}
    for p in connection.features.ports:
      self.numero[p.name] = p.port_no

    self.port_hote = None
    self.port_haut = None
    self.vers_voisin = {}
    if self.dpid in LIEN_HAUT:
      self.port_hote = self.port('eth1')
      for voisin, suffixe in VOISINS[self.dpid].items():
        self.vers_voisin[voisin] = self.port(suffixe)
      self.port_haut = self.vers_voisin[LIEN_HAUT[self.dpid]]
      log.info("s%i : hote=%s haut=%s voisins=%s", self.dpid, self.port_hote,
              self.port_haut, self.vers_voisin)
    else:
      log.info("s%i : commutateur a apprentissage classique", self.dpid)


  def port (self, suffixe):
    return self.numero['s%i-%s' % (self.dpid, suffixe)]
```

```
def resend_packet (self, packet_in, out_port):
  """
  Instructs the switch to resend a packet that it had sent to us.
  "packet_in" is the ofp_packet_in object the switch had sent to the
  controller due to a table-miss.
  """
  msg = of.ofp_packet_out()
  msg.data = packet_in

  # Add an action to send to the specified port
  action = of.ofp_action_output(port = out_port)
  msg.actions.append(action)

  # Send message to switch
  self.connection.send(msg)


def act_like_hub (self, packet, packet_in):
  """
  Implement hub-like behavior -- send all packets to all ports besides
  the input port.
  """

  # We want to output to all ports -- we do that using the special
  # OFPP_ALL port as the output port. (We could have also used
  # OFPP_FLOOD.)
  self.resend_packet(packet_in, of.OFPP_ALL)

  # Note that if we didn't get a valid buffer_id, a slightly better
  # implementation would check that we got the full data before
  # sending it (len(packet_in.data) should be == packet_in.total_len)).


def installer_flux (self, in_port, dst, out_port):
  msg = of.ofp_flow_mod()
  msg.match = of.ofp_match(in_port = in_port, dl_dst = dst)
  msg.idle_timeout = IDLE_TIMEOUT
  msg.hard_timeout = HARD_TIMEOUT
  if out_port is not None:
    msg.actions.append(of.ofp_action_output(port = out_port))
  self.connection.send(msg)


def apprendre_et_inonder (self, packet, packet_in):
  self.mac_to_port[packet.src] = packet_in.in_port
  if not packet.dst.is_multicast and packet.dst in self.mac_to_port:
    out_port = self.mac_to_port[packet.dst]
    if out_port != packet_in.in_port:
      self.resend_packet(packet_in, out_port)
  else:
    self.resend_packet(packet_in, of.OFPP_ALL)


def act_less_stupidly (self, packet, packet_in):
  """
```

```
    Implement a more intelligent behavior for the switch to take benefit of the multi
        routes
    """
    if self.dpid not in LIEN_HAUT:
      self.apprendre_et_inonder(packet, packet_in)
      return

    entree = packet_in.in_port
    dst = str(packet.dst)
    connu = dst in HOTES

    if packet.dst.is_multicast or not connu:
      if entree == self.port_hote:
        sortie = self.port_haut
      elif entree == self.port_haut:
        sortie = self.port_hote
      else:
        sortie = None
      if packet.dst.is_multicast:
        self.installer_flux(entree, packet.dst, sortie)
    else:
      switch_dest = HOTES[dst]
      if switch_dest == self.dpid:
        sortie = self.port_hote
      elif entree != self.port_hote:
        sortie = None
      elif switch_dest in self.vers_voisin:
        sortie = self.vers_voisin[switch_dest]
      else:
        sortie = self.port_haut
      self.installer_flux(entree, packet.dst, sortie)

    if sortie is not None:
      self.resend_packet(packet_in, sortie)


  def _handle_PacketIn (self, event):
    """
    Handles packet in messages from the switch.
    """

    packet = event.parsed # This is the parsed packet data.
    if not packet.parsed:
      log.warning("Ignoring incomplete packet")
      return

    packet_in = event.ofp # The actual ofp_packet_in message.

    # Comment out the following line and uncomment the one after
    # when starting the exercise.
    #self.act_like_hub(packet, packet_in)
    self.act_less_stupidly(packet, packet_in)



def launch ():
  """
  Starts the component
```

```
  """
  def start_switch (event):
    log.debug("Controlling %s" % (event.connection,))
    Diamond(event.connection)
  core.openflow.addListenerByName("ConnectionUp", start_switch)
```

B Modifications de `diamond.py` et `passerelle.py`

Les lignes précédées de `-` sont celles du fichier fourni, les lignes précédées de `+` sont les nouvelles. Les fichiers complets sont dans l'archive jointe.

`diamond.py`

```
-cm = CustomPOX ('pox', ip='127.0.0.1', port=6633)
-c = Controller ('c', port=6633)
+cm = CustomPOX ('pox', ip='127.0.0.1', port=6633,
+ cargs='openflow.of_01 --port=%s misc.tp')
+c = Controller ('c', port=6634)

-cmap = {'s1': cm, 's2': cm, 's3':cm, 's4' : c}
+cmap = {'s1': c, 's2': c, 's3': cm, 's4': cm}

-info( "*** Add defaults route to host1 and host2\n" )
-h1.cmd("ip route add 10.0.20.0/24 dev eth0")
-h2.cmd("ip route add 10.0.10.0/24 dev eth0")
+info( "*** Add routes to the other host subnets\n" )
+for i, h in enumerate([h1, h2, h3, h4], 1):
+ for j in range(1, 5):
+ if j != i:
+ h.cmd('ip route add 10.0.%d0.0/24 dev %s' % (j, h.defaultIntf().name))

-net.pingAll()
+#net.pingAll()
```

`passerelle.py`

```
-c1 = CustomPOX ('pox', ip='127.0.0.1', port=6633)
-c2 = Controller ('c2', port=6633)
+c1 = CustomPOX ('pox', ip='127.0.0.1', port=6633,
+ cargs='openflow.of_01 --port=%s forwarding.l3_learning --fakeways
    =10.0.10.254,10.0.30.254')
+c2 = Controller ('c2', port=6634)
```